

Session : Principale
Régime : RM
Enseignant : Sofiane HACHICHA
Classe(s) : CPI2
Barème : 14 + 6

Date : 06/01/2024
Matière : Programmation OO Java
Durée : 1h30mn
Documents : Non autorisés
Nombre des pages : 4

Examen

Exercice 1

Nous voulons développer un programme Java qui permet de gérer la location de biens immobiliers dans la ville de Gabès. Le programme est composé de deux paquets : **biens** et **locations**.

Partie A :

Le paquetage **biens** contient les classes **Immobilier**, **Villa** et **Appartement**.

Nous considérons que la ville de Gabès est subdivisée en 5 zones tarifaires numérotées de 1 à 5. Le tarif de location du mètre carré d'un bien immobilier dépend de la zone tarifaire où il se trouve. Les tarifs de toutes les zones de la ville de Gabès sont enregistrés dans un tableau statique de valeurs réelles appelé **tarifsM2**, lequel est défini dans la classe **Immobilier**. Ainsi, si ce tableau contient par exemple les valeurs {9.6, 7.2, 5.4, 4.1, 2.5}, alors le tarif de location du mètre carré d'un bien immobilier situé dans la zone 2 est 7.2 dinars.

Un bien immobilier peut être soit une villa, soit un appartement.

Par ailleurs, un bien immobilier (issu de la classe abstraite **Immobilier**) est caractérisé par :

- Une adresse (de type chaîne de caractères).
- Une zone (de type entier).
- Une superficie (de type réel).
- Un tableau statique de réels : **tarifsM2**.
- Un constructeur qui initialise uniquement les champs suivants : adresse, zone et superficie.
- Un getter pour l'attribut adresse.
- Un getter pour l'attribut superficie.
- Un setter pour le tableau **tarifsM2**.
- Une méthode **double tarifM2()** qui renvoie le tarif du mètre carré pour l'objet courant.
- Une méthode **void modifierTarifM2(double tarif, int n)** qui modifie le tarif associé à la zone spécifiée n.
- Une méthode abstraite **void afficherDetails()**. Elle serait utilisée pour afficher les détails spécifiques de chaque type d'immobilier.

Chaque Villa (une classe concrète qui hérite de la classe **Immobilier**) est caractérisée par :

- Une superficie Jardin (de type réel).
- Un getter pour cet attribut.

Chaque Appartement (une classe concrète qui hérite de la classe Immobilier) est caractérisé par :

- Un numéro d'étage (de type entier).
- Un code (de type chaîne de caractères).
- Un getter pour l'attribut numéro d'étage.
- Un getter pour l'attribut code.

Travail demandé :

1. Définir la classe Immobilier qui respecte le principe d'encapsulation.

package biens;

```
public abstract class Immobilier {  
    private String adresse;  
    private int zone;  
    private double superficie;  
    private static double[] tarifsM2={9.6, 7.2, 5.4, 4.1, 2.5};  
    public Immobilier(String adresse, int zone, double superficie) {  
        this.adresse= adresse;  
        this.zone=zone;  
        this.superficie= superficie;  
    }  
    public String getAdresse() {  
        return adresse;  
    }  
    public double getSuperficie() {  
        return superficie;  
    }  
    public void setTarifsM2(double[] val) {  
        tarifsM2=val;  
    }  
    public double tarifM2() {  
        if(zone==1)  
            return tarifsM2[0];  
        else if(zone==2)  
            return tarifsM2[1];  
        else if(zone==3)  
            return tarifsM2[2];  
        else if(zone==4)  
            return tarifsM2[3];  
        else  
            return tarifsM2[4];  
    }  
    public void modifierTarifM2(double tarif, int n) {  
        if(zone ==n)  
            tarifsM2[n-1]=tarif;  
    }  
    public abstract void afficherDetails();  
}
```

2. Définir les classes Villa et Appartement qui respectent aussi le principe d'encapsulation.

```
package biens;
public class Villa extends Immobilier{
    private double superficie_jardin;
    public Villa(String adresse, int zone, double superficie, double superficie_jardin)
    {
        super(adresse,zone, superficie);
        this.superficie_jardin=superficie_jardin;
    }
    public double getSuperficieJardin() {
        return superficie_jardin;
    }
    public void afficherDetails() {
        System.out.println("Villa ayant un jardin de surface : "+getSuperficieJardin());
    }
}

package biens;
public class Appartement extends Immobilier{
    private int num;
    private String code;
    public Appartement(String adresse, int zone, double superficie, int num, String code) {
        super(adresse,zone, superficie);
        this.num=num;
        this.code=code;
    }
    public int getNum() {
        return num;
    }
    public String getCode() {
        return code;
    }
    public void afficherDetails() {
        System.out.println("Appartement ayant un code : "+getCode()+" et un numéro d'étage : "+getNum());
    }
}
```

Partie B :

Le paquetage **locations** contient l'interface scellée **IMaxLoyer** et les classes finales **MaxLoyerParticulier** et **MaxLoyerSociete** qui implémentent cette interface.

L'interface **IMaxLoyer** contient uniquement la méthode **double calculerMax(Immobilier m)**.

Tout bien immobilier **m** possède un montant de loyer maximum qu'on ne doit pas dépasser. La formule de calcul de ce montant varie selon que le bien va être loué à un particulier ou à une société.

S'il s'agit d'un **particulier**, la formule est la suivante :

$$\text{Loyer max de m} = \text{tarif du mètre carré} * (0.7 + (19 / \text{superficie})) * \text{superficie}$$

. S'il s'agit d'une **société**, la formule est la suivante :

$$\text{Loyer max de m} = 1.5 * (\text{tarif du mètre carré} * (0.7 + (19 / \text{superficie})) * \text{superficie})$$

Il est essentiel de mentionner que la superficie d'une villa est déterminée en ajoutant la superficie du jardin à la superficie de l'immobilier auquel elle est associée.

Travail demandé :

1. Définir l'interface IMaxLoyer.

```
package locations;
import biens.Immobilier;
public sealed interface IMaxLoyer permits MaxLoyerParticulier, MaxLoyerSociete {
    double calculerMax(Immobilier m);
}
```

2. Définir les classes MaxLoyerParticulier et MaxLoyerSociete.

```
package locations;
import biens.*;
public final class MaxLoyerParticulier implements IMaxLoyer {
    public double calculerMax(Immobilier m) {
        if(m instanceof Villa)
            return m.tarifM2()*(0.7+ (19/(m.getSuperficie() +
                ((Villa) m).getSuperficieJardin())) *
                (m.getSuperficie() + ((Villa) m).getSuperficieJardin()));
        return m.tarifM2()*(0.7+ (19/m.getSuperficie()))*m.getSuperficie();
    }
}
```

```
package locations;
import biens.*;
public final class MaxLoyerSociete implements IMaxLoyer {
    public double calculerMax(Immobilier m) {
        if(m instanceof Villa)
            return 1.5 *(m.tarifM2()*(0.7+(19/(m.getSuperficie() +
                ((Villa) m).getSuperficieJardin()))*
                (m.getSuperficie() + ((Villa) m).getSuperficieJardin()));
        return 1.5*(m.tarifM2()*(0.7+(19/m.getSuperficie()))*m.getSuperficie());
    }
}
```

Partie C :

Le paquetage **locations** contient aussi les classes **Location** et **GestionLocations**.

Chaque Location est caractérisée par :

- Un bien loué (de type Immobilier).
- Un locataire (de type chaîne de caractères).
- Un maxLoyer qui définit la formule à utiliser (particulier ou société) pour calculer le montant du loyer maximum (de type IMaxLoyer).
- Un montant du loyer (de type réel).
- Un constructeur qui initialise uniquement le bien loué, le locataire et le maxLoyer.
- Des getters pour tous les attributs.
- Une méthode **void affecterMontantMax()** qui associe le montant du loyer à la valeur maximale calculée à partir l'objet maxLoyer.
- Une méthode **void setMontantLoyer(double val)** qui associe le montant du loyer à la valeur val donnée en paramètre. Si la valeur de val est supérieure au montant du loyer maximal, le montant du loyer prend la valeur du montant du loyer maximal.

La classe GestionLocations définit :

- Un tableau statique privé de type **Location** nommé **lesLocations** de taille maximale de 1000 éléments.
- Un attribut statique privé nommé **nbLocations** qui sert à compter le nombre d'éléments effectifs dans le tableau lesLocations.
- Une méthode statique **void ajouterLocation(Immobilier m, String locataire, double montantLoyer, IMaxLoyer maxLoy)** qui crée une nouvelle location à partir des paramètres et l'ajoute au tableau. Si le montantLoyer fourni en paramètre est supérieur au montant maximal, le montant du loyer est fixé au montant maximal pour cette location ; sinon, le montant du loyer prend la valeur du paramètre. Il est important de souligner que le tableau lesLocations peut contenir des doublons (des locations répétées).
- Une méthode statique **int nbLocationsMax()** qui renvoie le nombre de locations du tableau lesLocations dont le bien loué est un appartement au rez-de-chaussée (étage 0) et le montant du loyer est égal au maximum. Cette méthode recherche les locations qui correspondent à ces critères dans le tableau lesLocations et renvoie le nombre correspondant.

Travail demandé :

1. Définir les classes Location qui respecte le principe d'encapsulation.

```
package locations;
import biens.*;
public class Location {
    private Immobilier bienLoue;
    private String locataire;
    private IMaxLoyer maxLoyer;
    private double montantLoyer;

    public Location(Immobilier bienLoue, String locataire, IMaxLoyer maxLoyer) {
        this.bienLoue = bienLoue;
        this.locataire=locataire;
        this.maxLoyer=maxLoyer;
    }
}
```

```

    public String getLocataire() {
        return locataire;
    }

    public double getMontantLoyer() {
        return montantLoyer;
    }

    public Immobilier getBienLoue() {
        return bienLoue;
    }

    public IMaxLoyer getMaxLoyer() {
        return maxLoyer;
    }

    public void affecterMontantMax() {
        montantLoyer = maxLoyer.calculerMax(bienLoue);
    }

    public void setMontantLoyer(double val) {
        if(val > maxLoyer.calculerMax(bienLoue))
            affecterMontantMax();
        else
            montantLoyer=val;
    }
}

```

2. Définir la classe GestionLocations.
3. Ajouter une méthode main dans la classe principale GestionLocations pour effectuer des opérations spécifiques : créer une villa et afficher ses détails. Ensuite, la louer à un particulier, ajouter cette location au tableau lesLocations, et enfin, afficher des informations telles que le locataire et le montant du loyer.

```

package locations;
import biens.*;
public class GestionLocations {

    private static Location[] lesLocations=new Location[100];
    private static int nbLocations=0;
    // public GestionLocations() {
    //     lesLocations=new Location[100];
    //     nbLocations=0;
    // }
    public static void ajouterLocation(Immobilier m, String locataire, double
montantLoyer, IMaxLoyer maxLoy) {
        lesLocations[nbLocations] = new Location(m, locataire,maxLoy);

        System.out.println(lesLocations[nbLocations].getMaxLoyer().calculerMax(m));
        lesLocations[nbLocations].setMontantLoyer(montantLoyer);
        nbLocations++;
    }
}

```

```

    public static int nbLocationsMax() {
        int res=0;
        for(int i=0; i<nbLocations; i++) {
            if(lesLocations[i].getBienLoue() instanceof Appartement &&
lesLocations[i].getMontantLoyer() ==
lesLocations[i].getMaxLoyer().calculerMax(lesLocations[i].getBienLoue()) )
                if(((Appartement) lesLocations[i].getBienLoue()).getNum() ==0)
                    res++;
        }
        return res;
    }

    public static void main(String[] args) {
        Immobilier v = new Villa("Paradis", 1, 100, 20.5);
        v.afficherDetails();
        IMaxLoyer im = new MaxLoyerParticulier();
        ajouterLocation(v, "mohamed", 1023.5, im);

        for(int j=0; j<nbLocations; j++) {
            System.out.println("Locataire : "+lesLocations[j].getLocataire());
            System.out.println("Montant du loyer : "+lesLocations[j].getMontantLoyer());
        }
    }
}

```

Exercice 2

1. Définir une interface fonctionnelle **ICompare** ayant une méthode abstraite nommée **inferieur** prenant **deux objets** en paramètres et retournant un **booléen**. Cette méthode doit comparer les deux objets et renvoyer true si le premier objet est inférieur au deuxième, et false dans le cas contraire.

```

interface ICompare {
    boolean inferieur(Object a, Object b);
}

```

2. Définir une classe **TriGenerique** ayant une méthode statique nommée **ranger** prenant un **tableau d'objets** et un objet de type **ICompare** en paramètres. Cette méthode doit trier le tableau d'objets en utilisant l'objet ICompare pour comparer les éléments du tableau.

```

class TriGenerique {
    public static void ranger(Object[] tab, ICompare obj) {
        for (int i = 0; i < tab.length; i++) {
            for (int j = 1; j < tab.length - i; j++) {
                if (!obj.inferieur(tab[j - 1], tab[j])) {
                    Object x = tab[j - 1];
                    tab[j - 1] = tab[j];
                    tab[j] = x;
                }
            }
        }
    }
}

```

```

    }
}
}

```

3. Définir un **record** nommé **Personne** ayant deux attributs :
- **nom** : une chaîne de caractères représentant le nom de la personne.
 - **age** : un entier représentant l'âge de la personne.

Ce record contient également un constructeur avec un seul paramètre qui initialise le nom de la personne au paramètre fourni et fixe l'âge de la personne à zéro.

```

record Personne(String nom, int age) {
    public Personne(String nom) {
        this(nom,0);
    }
}

```

4. Définir une classe principale **TestPersonne** contenant la méthode main qui permet de :
- Créer un tableau d'objets de type **Personne** contenant trois personnes.
 - Déclarer et initialiser une variable de type **ICompare** en utilisant une expression lambda pour que la méthode inferieur doit comparer les personnes par leur âge. Cette méthode doit renvoyer true si la première personne est plus jeune que la deuxième, et false dans le cas contraire.
 - Invoquer la méthode ranger de la classe TriGenerique en passant le tableau de type Personne et la variable de type ICompare en paramètres.
 - Parcourir le tableau de personnes triées selon leur âge et afficher les caractéristiques de chaque personne.

```

public class TestPersonne {
    public static void main(String[] args) {
        Personne[] personnes = new Personne[3];
        personnes[0] = new Personne("Ali", 25);
        personnes[1] = new Personne("Olfa", 20);
        personnes[2] = new Personne("Sami");
        ICompare comparer = (Object obj1, Object obj2) ->
        {
            Personne p1 = (Personne) obj1;
            Personne p2 = (Personne) obj2;
            return p1.age() < p2.age();
        };

        TriGenerique.ranger(personnes, comparer);

        for(Personne p : personnes)
            System.out.println(p);
    }
}

```